Technical Project Study Notes

# Cab Routing and Dynamic Fare Allocation

Project Walkthrough, Implementation Review, and Interview Preparation

---

**What this document contains**

Project motivation and objective; graph representation and route costs; system architecture; detailed explanations of the C++ implementation; Dijkstra's shortest-path algorithm; seat-capacity and spare-seat logic; sample input/output walkthrough; complexity analysis; implementation limitations and recommended fixes; and an interview question bank.

---

| | |
|---|---|
| **Implementation language:** | C++ (requires C++17 for structured bindings) |
| **Core graph algorithm:** | Dijkstra's shortest-path algorithm |
| **Graph representation:** | Adjacency list using nested unordered maps |
| **Capacity model:** | Four seats per active cab, plus configured spare seats |
| **Input / output:** | Text files: `input.txt` and `output.txt` |

Prepared as a study guide for technical interviews

Based on the supplied C++ source, input file, and simulation output

# Contents

**1 Executive Summary** **1**
1.1 What is implemented, simulated, and missing? . . . 1
1.2 The main technical caveat . . . 1

**2 Project Motivation and Problem Understanding** **1**
2.1 What problem does the project address? . . . 1
2.2 What is a weighted graph? . . . 2
2.3 How is route cost calculated? . . . 2
2.4 What does capacity mean here? . . . 2

**3 End-to-End System Architecture** **2**
3.1 Processing pipeline . . . 3
3.2 Major classes and functions . . . 3

**4 Detailed Walkthrough of the Code** **3**
4.1 Packages and imports . . . 3
4.2 Edge structure . . . 4
4.3 Custom hash and normalized route key . . . 4
4.4 CabSystem data members . . . 5
4.5 Adding a route . . . 5
4.6 Registering inactive-route spare seats . . . 5
4.7 Dijkstra's shortest-path algorithm . . . 5
4.7.1 Data structures used . . . 5
4.7.2 Relaxation step . . . 6
4.7.3 Path reconstruction . . . 6
4.8 Spare-seat assignment . . . 6
4.9 Dynamic fare calculation . . . 6
4.10 Input parser and file processing . . . 7
4.11 Main function . . . 7

**5 Input Data and Graph Representation** **8**
5.1 Input file . . . 8
5.2 Route interpretation . . . 8
5.3 Graph diagram . . . 8

**6 Worked Example: Understanding the Output** **9**
6.1 Program output . . . 9
6.2 Request 1: A to B, 10 passengers . . . 9
6.3 Request 2: A to C, 3 passengers . . . 9
6.4 Request 3: B to C, 7 passengers . . . 9
6.5 Request 4: C to D, 5 passengers . . . 9

6.6 Output summary . . . 10

**7 Algorithms and Complexity Analysis 10**

7.1 Dijkstra's algorithm . . . 10

7.1.1 Pseudocode . . . 10

7.2 Time complexity . . . 10

7.3 Why use an adjacency list? . . . 11

**8 Important Implementation Issues and Recommended Fixes 11**

8.1 A more reliable design direction . . . 12

**9 Interview Questions and Suggested Answers 12**

9.1 Project overview and motivation . . . 12

9.1.1 Q1. Explain your project in one minute. . . . 12

9.1.2 Q2. What problem are you trying to solve? . . . 13

9.1.3 Q3. Why did you represent the city as a graph? . . . 13

9.1.4 Q4. Is your graph directed or undirected? . . . 13

9.2 Graph algorithms and data structures . . . 13

9.2.1 Q5. Which shortest-path algorithm did you use and why? . . . 13

9.2.2 Q6. How do you calculate the weight of an edge? . . . 13

9.2.3 Q7. Why did you use a priority queue? . . . 13

9.2.4 Q8. What is the purpose of the parent map? . . . 13

9.2.5 Q9. What is the time complexity of Dijkstra in your implementation? . . . 13

9.2.6 Q10. Why use an adjacency list instead of an adjacency matrix? . . . 13

9.3 C++ and object-oriented design . . . 14

9.3.1 Q11. Why did you create an Edge structure? . . . 14

9.3.2 Q12. Why use unordered_map? . . . 14

9.3.3 Q13. Why is a custom hash required? . . . 14

9.3.4 Q14. Why does make_key() sort the two endpoints? . . . 14

9.3.5 Q15. Which C++ standard does the code require? . . . 14

9.4 Capacity and fare logic . . . 14

9.4.1 Q16. How do you calculate active seat capacity? . . . 14

9.4.2 Q17. Why do you take the minimum capacity along the path? . . . 14

9.4.3 Q18. Explain your fare calculation. . . . 14

9.4.4 Q19. Are spare seats allocated to the selected route only? . . . 14

9.4.5 Q20. Does the program update active seat capacity after a booking? . . . 15

9.4.6 Q21. Is the returned fare multiplied by the number of passengers? . . . 15

9.5 Input/output and limitations . . . 15

9.5.1 Q22. How does the program read the input file? . . . 15

9.5.2 Q23. What happens when no path exists? . . . 15

9.5.3 Q24. What are the main limitations? . . . 15

9.5.4 Q25. How would you improve the project? . . . 15

9.5.5 Q26. Can this be called a production cab-booking system? . . . 15

**10 Interview Cheat Sheet** **15**
10.1 Key numbers to remember . . . . . 16
10.2 Quick vocabulary . . . . . 16

**11 A Strong and Honest Final Project Pitch** **16**
11.1 Final checklist before an interview . . . . . 16

**A Complete Supplied C++ Source Code** **17**

# 1 Executive Summary

The project is a C++ simulation of a cab-routing system over a network of locations. It represents locations as vertices and direct routes as weighted, undirected edges. Each edge stores a base travel cost, a congestion cost, the number of active cabs, and the resulting seat capacity. Passenger requests are read from a text file. For each request, the program finds a minimum-cost path using Dijkstra's algorithm, determines the bottleneck active seat capacity along that path, and calculates a fare using a simple capacity-dependent multiplier.

The implementation also stores spare seats against inactive routes. When a request exceeds the active capacity of its selected path, the program checks the total spare-seat pool. It applies a 25% fare increase when the pool is sufficient and a 50% increase otherwise.

**One-sentence summary:** The project combines graph-based minimum-cost routing with a basic capacity-aware fare adjustment and a text-file simulation of passenger requests. The current implementation is a prototype: spare seats are pooled globally rather than matched to the requested path, and active seat occupancy is not tracked between requests.

## 1.1 What is implemented, simulated, and missing?

| Area | Status in the supplied code |
|---|---|
| Graph construction | Implemented: routes are stored in an undirected adjacency-list structure. |
| Route cost | Implemented: an edge cost is calculated as base cost plus congestion cost. |
| Shortest path | Implemented: Dijkstra's algorithm selects a minimum-cost path. |
| Active capacity | Implemented as a static capacity: each active cab contributes four seats. |
| Spare-seat storage | Implemented: inactive-route spare seats are stored by an unordered route key. |
| Spare-seat assignment | Implemented, but globally: it does not verify that spare seats belong to the requested path. |
| Dynamic fare | Implemented: base route cost is multiplied by 1.00, 1.25, or 1.50 based on the capacity check. |
| Request processing | Implemented: requests are parsed from a text file and results are written to another file. |
| Live cab tracking | Not implemented: occupied seats and cab positions are not updated. |
| Production booking service | Not implemented: there is no real-time dispatch, network API, persistence layer, or concurrency control. |

## 1.2 The main technical caveat

The shortest-path function optimizes route cost only. Capacity is checked after the path has already been selected. In addition, spare seats from any inactive route can be used to cover a shortage on any selected path. Therefore, the current code should be described as a route-cost and fare simulation, not as a fully constrained vehicle-dispatch optimizer.

# 2 Project Motivation and Problem Understanding

## 2.1 What problem does the project address?

A cab-routing system must choose a route between a passenger's pickup and destination locations. Different direct routes may have different operating costs and congestion levels. A routing algorithm

can compare these costs and select the minimum-cost path. A booking system must also consider whether enough seats are available to serve the requested number of passengers.

This project models those two concerns in a simplified way:

1. Find a path with the lowest sum of route costs.
2. Estimate the active seat capacity available along that path.
3. Use configured spare seats when the active capacity is not sufficient.
4. Increase the fare when the request requires additional capacity or when spare capacity is insufficient.

### 2.2 What is a weighted graph?

A graph is a mathematical structure consisting of vertices and edges. In this project:

- A vertex represents a location, such as A, B, C, or D.
- An edge represents a direct route between two locations.
- The edge weight represents the route's combined base and congestion cost.

The graph is undirected because the `add_route()` function inserts each route in both orientations. Thus, a route from A to B is also available from B to A with the same stored properties.

### 2.3 How is route cost calculated?

For an edge from location $u$ to location $v$:

$$w(u, v) = \text{base_cost}(u, v) + \text{congestion_cost}(u, v).$$

For a path containing multiple edges, the path cost is the sum of the edge costs:

$$C(P) = \sum_{(u,v)\in P} w(u, v).$$

The Dijkstra implementation minimizes this sum. The cost is not multiplied by the number of passengers in the supplied code; it is used as the route-level fare basis.

### 2.4 What does capacity mean here?

The constructor of `Edge` assumes that every active cab has four seats:

$$\text{active seats on edge} = 4 \times \text{number of active cabs}.$$

For a multi-edge path, the implementation uses the minimum edge capacity as the path's bottleneck capacity. This is a simplified approximation: it assumes that the same passenger group must be supported by every edge on the route.

## 3 End-to-End System Architecture

### 3.1 Processing pipeline

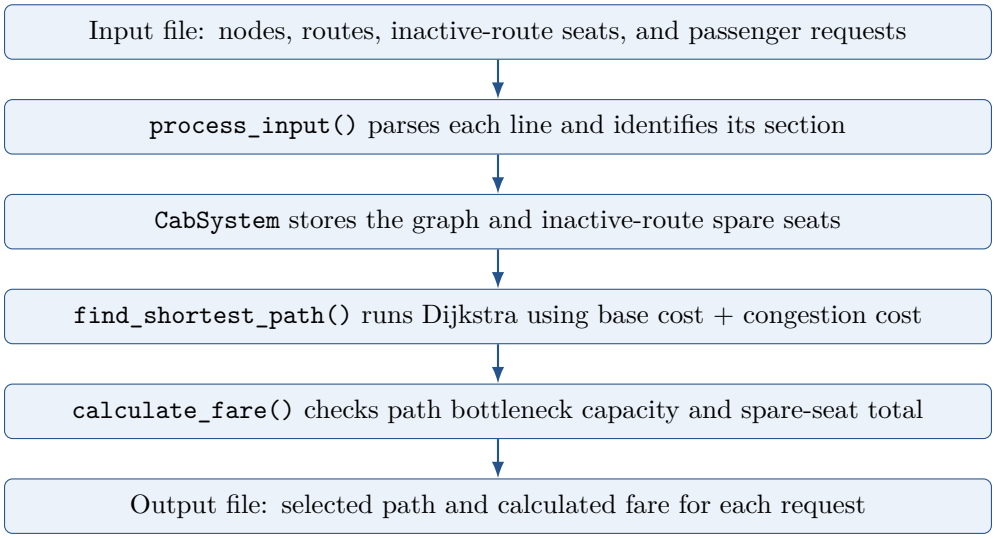



Figure 1: High-level flow implemented by the supplied program.

### 3.2 Major classes and functions

| Class / function | Responsibility |
|---|---|
| `Edge` | Stores base cost, congestion cost, active cab count, and active seat capacity for one direct route. |
| `pair_hash` | Supplies a hash function so a pair of strings can be used as an unordered-map key. |
| `make_key()` | Normalizes a pair of locations into a consistent order, so A–B and B–A share one inactive-route key. |
| `CabSystem::add_route()` | Inserts an undirected route into the graph. |
| `CabSystem::set_inactive_route()` | Stores the spare-seat count for an inactive route. |
| `CabSystem::find_shortest_path()` | Finds a minimum-cost path and reconstructs its sequence of locations. |
| `CabSystem::assign_spare_seats()` | Consumes spare seats from the global inactive-route map. |
| `CabSystem::calculate_fare()` | Applies the path-capacity check and fare multiplier. |
| `process_input()` | Reads sections from the input file, processes requests, and writes results. |
| `main()` | Creates the cab-system object and starts file processing. |

## 4 Detailed Walkthrough of the Code

### 4.1 Packages and imports

The source includes standard C++ headers for console I/O, file I/O, string streams, hash maps, vectors, strings, priority queues, numeric limits, and algorithms.

| Header | Purpose in this program |
|---|---|
| `iostream` | Prints the completion message to the console. |
| `fstream` | Reads `input.txt` and writes `output.txt`. |
| `sstream` | Parses whitespace-separated fields from each input line. |
| `unordered_map` | Stores graph adjacency lists and inactive-route capacities. |
| `vector` | Stores the reconstructed path. |
| `string` | Represents location labels. |
| `queue` | Provides the priority queue used by Dijkstra's algorithm. |
| `limits` | Provides the maximum representable double value. |
| `algorithm` | Provides functions such as `min()` and `reverse()`. |

The source uses `INT_MAX` and `greater<>`/hash utilities. For portable, explicit includes, add `#include <climits>` for `INT_MAX` and `#include <functional>` for standard functional utilities. The code also uses structured bindings, so compile it with C++17 or later.

## 4.2 Edge structure

The `Edge` structure groups the information associated with a direct route.

```
struct Edge {
    double base_cost;
    double congestion_cost;
    int cars;
    int seats;

    Edge(double base = 0, double congestion = 0, int cabs = 0)
        : base_cost(base), congestion_cost(congestion),
          cars(cabs), seats(cabs * 4) {}
};
```

The constructor initializes the four fields. The seat capacity is calculated using a fixed assumption of four seats per cab. The `cars` field is stored, while the capacity checks later use the derived `seats` field.

## 4.3 Custom hash and normalized route key

The inactive routes are stored using a key of type `pair<string,string>`. An unordered map requires a hash function for its key type, so the program defines `pair_hash`.

```
struct pair_hash {
    size_t operator()(const pair<string, string>& p) const {
        return hash<string>()(p.first) ^
               (hash<string>()(p.second) << 1);
    }
};

pair<string, string> make_key(const string& a, const string& b) {
    return a < b ? make_pair(a, b) : make_pair(b, a);
}
```

The XOR-based expression produces a hash value from both strings. Hash collisions are permitted by an unordered map; the map also checks key equality, so a collision does not mean two different keys are treated as equal. The `make_key()` function ensures that the same undirected route uses one canonical key regardless of the order in which its endpoints are supplied.

## 4.4 CabSystem data members

```
class CabSystem {
private:
    unordered_map<string, unordered_map<string, Edge>> graph;
    unordered_map<pair<string, string>, int, pair_hash> inactive_routes;
};
```

The `graph` is a nested unordered map. The outer key is a location; the inner map stores its neighbouring locations and the corresponding `Edge`. This is an adjacency-list representation. The `inactive_routes` map stores spare-seat counts using normalized route pairs.

## 4.5 Adding a route

```
void add_route(const string& a, const string& b,
               double base, double congestion, int cars) {
    Edge e(base, congestion, cars);
    graph[a][b] = e;
    graph[b][a] = e;
}
```

A single `Edge` object is created and copied into both adjacency directions. This makes the graph undirected. The cost and capacity are identical in both directions in the current model.

## 4.6 Registering inactive-route spare seats

```
void set_inactive_route(const string& a, const string& b,
                        int spare_seats) {
    inactive_routes[make_key(a, b)] = spare_seats;
}
```

The function uses `make_key()` before inserting the spare-seat count. This prevents the same undirected route from being represented separately as (A,B) and (B,A). However, later allocation logic sums spare seats across all entries and does not match an entry to the selected path.

## 4.7 Dijkstra's shortest-path algorithm

The function `find_shortest_path(src,dest)` returns a pair containing the minimum route cost and the path as a vector of location strings.

### 4.7.1 Data structures used

| Variable | Role |
|---|---|
| `dist` | Stores the best distance currently known from the source to each location. |
| `parent` | Stores the predecessor location used to reconstruct the selected path. |
| `pq` | A min-priority queue of (distance, location) pairs. |
| `path` | Stores the final ordered sequence from source to destination. |

```
using PQ = pair<double, string>;
priority_queue<PQ, vector<PQ>, greater<PQ>> pq;
pq.emplace(0, src);
```

The comparator `greater<PQ>` makes the priority queue return the smallest distance first. This is the ordering needed by Dijkstra's algorithm.

### 4.7.2 Relaxation step

For each neighbour `v` of the current location `u`, the code calculates:

$$w(u, v) = e.\text{base_cost} + e.\text{congestion_cost}.$$

If going through `u` gives a smaller distance to `v`, it updates `dist[v]`, stores `u` as the parent of `v`, and pushes the new pair into the priority queue.

```
double cost = e.base_cost + e.congestion_cost;
if (dist[v] > dist[u] + cost) {
    dist[v] = dist[u] + cost;
    parent[v] = u;
    pq.emplace(dist[v], v);
}
```

Dijkstra's algorithm requires non-negative edge weights. The supplied sample has positive weights. The implementation does not validate that constraint, so negative route costs should be rejected by input validation or handled with a different shortest-path algorithm.

### 4.7.3 Path reconstruction

After the search, the function starts at the destination and repeatedly follows the `parent` map until it reaches the source. It then reverses the collected sequence. If a predecessor is missing, it returns an empty path, which is used by the caller to identify an unreachable destination.

> The algorithm selects the cheapest path based on cost only. It does not consider the seat capacity of candidate paths while searching. Capacity is checked after Dijkstra has chosen a path.

## 4.8 Spare-seat assignment

```
int assign_spare_seats(int required) {
    int assigned = 0;
    for (auto& [route, spare] : inactive_routes) {
        int now = min(required - assigned, spare);
        spare -= now;
        assigned += now;
        if (assigned >= required) break;
    }
    return assigned;
}
```

The function iterates over the inactive-route map and consumes seats until the requested number is assigned or no more entries are available. The spare-seat counts are reduced, so this resource is consumed across later requests. Since the container is an `unordered_map`, its iteration order is not a guaranteed business priority. More importantly, the code does not check whether the inactive route is part of the passenger's selected path.

## 4.9 Dynamic fare calculation

The `calculate_fare()` function performs four steps:

1. Calls Dijkstra to obtain the minimum-cost path.
2. Finds the minimum active seat capacity across every edge in that path.
3. Compares the passenger count with that bottleneck capacity.
4. Applies the corresponding fare multiplier and consumes spare seats if needed.

For a path $P$ with active edge capacities $S_1, S_2, \ldots, S_k$, the code calculates:

$$S_{\text{path}} = \min(S_1, S_2, \ldots, S_k).$$

Let $P_c$ be the number of passengers and $C(P)$ the path cost. The current pricing rule is:

$$F = \begin{cases} C(P), & P_c \le S_{\text{path}}, \\ 1.25\,C(P), & P_c > S_{\text{path}} \text{ and spare total is sufficient}, \\ 1.50\,C(P), & P_c > S_{\text{path}} \text{ and spare total is insufficient}. \end{cases}$$

```
if (P <= seat_cap) return base_cost;

int unfilled = P - seat_cap;
int spare_total = 0;
for (const auto& [_, spare] : inactive_routes)
    spare_total += spare;

if (spare_total >= unfilled) {
    assign_spare_seats(unfilled);
    return 1.25 * base_cost;
} else {
    assign_spare_seats(spare_total);
    return 1.5 * base_cost;
}
```

The value returned is the route-cost basis multiplied by a tier. It is not calculated as a per-seat price and is not multiplied by the passenger count in the current code. Also, when spare capacity is insufficient, the code still returns a fare after assigning all spare seats; it does not reject the request or report how many passengers remain unaccommodated.

### 4.10 Input parser and file processing

The parser reads the file line by line and tracks the current section using a string named `section`. It recognizes four section headers: `NODES:`, `ROUTES:`, `INACTIVE:`, and `REQUESTS:`.

- In the `routes` section, it reads two location labels, base cost, congestion cost, and cab count, then calls `add_route()`.
- In the `inactive` section, it reads two location labels and a spare-seat count, then calls `set_inactive_route()`.
- In the `requests` section, it reads source, destination, and passenger count, then calls `calculate_fare()`.

Although the parser recognizes the `NODES:` header, it does not process the node names. The actual graph vertices are created when routes are inserted. This means a location with no route will not be added explicitly by the current parser.

### 4.11 Main function

```
int main() {
    CabSystem system;
    process_input(system, "input.txt", "output.txt");
    cout << "Simulation complete. Check output.txt for results." << endl;
    return 0;
}
```

The main function creates one `CabSystem` object and delegates the complete simulation to `process_input()`. The program uses fixed file names rather than command-line arguments.

# 5 Input Data and Graph Representation

## 5.1 Input file

The supplied input file is:

```
NODES: A B C D

ROUTES:
A B 10.0 2.0 2
A C 15.0 3.0 1
B C 8.0 1.5 2
C D 12.0 2.5 1

INACTIVE:
C D 4
A C 2

REQUESTS:
A B 10
A C 3
B C 7
C D 5
```

## 5.2 Route interpretation

Each route line has the structure:

`source destination base_cost congestion_cost active_cabs.`

Table 1: Active routes from the supplied input file

| Route | Base | Congestion | Combined cost | Cabs | Seats |
|---|---|---|---|---|---|
| A–B | 10.0 | 2.0 | 12.0 | 2 | 8 |
| A–C | 15.0 | 3.0 | 18.0 | 1 | 4 |
| B–C | 8.0 | 1.5 | 9.5 | 2 | 8 |
| C–D | 12.0 | 2.5 | 14.5 | 1 | 4 |

The inactive-route section configures four spare seats for C–D and two for A–C. The current allocation code treats these as a combined pool of six seats; it does not associate the seats with specific route segments during a booking.

## 5.3 Graph diagram

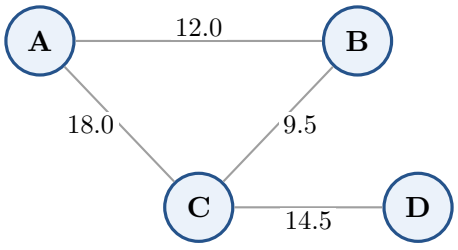



Figure 2: Undirected route graph. Edge labels show base cost plus congestion cost.

For example, there are two paths from A to C in this graph:

$$A \to C : 18.0,$$
$$A \to B \to C : 12.0 + 9.5 = 21.5.$$

Dijkstra chooses the direct route A–C because its total cost is lower.

# 6 Worked Example: Understanding the Output

## 6.1 Program output

The supplied `output.txt` contains:

```
Request: A -> B (10 passengers)
Path used: A -> B
Fare: 15

Request: A -> C (3 passengers)
Path used: A -> C
Fare: 18

Request: B -> C (7 passengers)
Path used: B -> C
Fare: 9.5

Request: C -> D (5 passengers)
Path used: C -> D
Fare: 18.125
```

## 6.2 Request 1: A to B, 10 passengers

The A–B edge has two active cabs, giving $2 \times 4 = 8$ active seats. The request needs ten seats, so the shortage is two. The configured global spare-seat pool initially contains $4 + 2 = 6$ seats. Because six is at least two, the program applies the 25% multiplier:

$$C(A, B) = 10 + 2 = 12, \qquad F = 1.25 \times 12 = \boxed{15}.$$

The program consumes two spare seats from the inactive-route map. Which map entry is reduced first is determined by unordered-map iteration order.

## 6.3 Request 2: A to C, 3 passengers

The direct A–C edge has one active cab and therefore four seats. Since three passengers fit within that capacity, no spare seats are needed and the base route cost is returned:

$$C(A, C) = 15 + 3 = \boxed{18}.$$

## 6.4 Request 3: B to C, 7 passengers

The B–C edge has two active cabs and therefore eight seats. Seven passengers fit within the active capacity:

$$C(B, C) = 8 + 1.5 = \boxed{9.5}.$$

## 6.5 Request 4: C to D, 5 passengers

The C–D edge has one active cab, giving four seats. The request needs one additional seat. The first request consumed two from the initial global pool of six, leaving four. The shortage of one is therefore covered according to the program's global-pool rule:

$$C(C, D) = 12 + 2.5 = 14.5, \qquad F = 1.25 \times 14.5 = \boxed{18.125}.$$

### 6.6 Output summary

Table 2: Verification of the supplied simulation output

| Request | Passengers | Selected path | Fare | Reason |
|---|---|---|---|---|
| A → B | 10 | A–B | 15.000 | Active capacity 8; shortage 2; global spare pool is sufficient. |
| A → C | 3 | A–C | 18.000 | Active capacity 4; no shortage. |
| B → C | 7 | B–C | 9.500 | Active capacity 8; no shortage. |
| C → D | 5 | C–D | 18.125 | Active capacity 4; shortage 1; remaining global spare pool is sufficient. |

The sample output is consistent with the current code's global spare-seat accounting. It does not prove that spare cabs are physically available on the selected route, because route matching is not performed by `assign_spare_seats()`.

## 7 Algorithms and Complexity Analysis

### 7.1 Dijkstra's algorithm

Dijkstra's algorithm is a greedy single-source shortest-path algorithm for graphs with non-negative edge weights. It repeatedly selects the unprocessed vertex with the smallest tentative distance and relaxes its outgoing edges. In this implementation, a min-priority queue is used to select the next vertex efficiently.

#### 7.1.1 Pseudocode

```
DIJKSTRA(graph, source, destination):
    for each vertex v:
        dist[v] = infinity
    dist[source] = 0
    parent = empty map
    pq = min-priority queue
    push (0, source) into pq

    while pq is not empty:
        (current_distance, u) = pop minimum from pq
        if u is destination:
            break

        for each edge (u, v):
            weight = base_cost(u, v) + congestion_cost(u, v)
            if dist[v] > dist[u] + weight:
                dist[v] = dist[u] + weight
                parent[v] = u
                push (dist[v], v) into pq

    reconstruct destination-to-source using parent
    reverse reconstructed path
    return dist[destination], path
```

### 7.2 Time complexity

Let $V$ be the number of locations, $E$ the number of unique undirected routes, $I$ the number of inactive-route entries, and $L$ the number of edges in the selected path.

| Operation | Time complexity / explanation |
|---|---|
| Graph construction | Expected $O(1)$ per hash-map insertion; $O(V+E)$ storage overall. |
| Dijkstra with adjacency list and binary heap | $O((V+E)\log V)$ in the standard implementation model. |
| Path reconstruction | $O(L)$, where $L$ is the number of edges in the selected path. |
| Bottleneck capacity scan | $O(L)$. |
| Spare-seat total calculation | $O(I)$. |
| Spare-seat allocation | $O(I)$ in the worst case. |
| One passenger request | $O((V+E)\log V + I + L)$, dominated by Dijkstra for typical sparse graphs. |

The program runs Dijkstra separately for every request. If there are $R$ requests, the overall shortest-path work is approximately $O(R(V+E)\log V)$, plus spare-seat scans and allocations. The memory usage is $O(V+E+I)$ for the graph and inactive-route map, excluding temporary per-request structures.

### 7.3 Why use an adjacency list?

An adjacency list stores only the neighbours that actually have a direct route. For a sparse road network, this is generally more space-efficient than a $V \times V$ adjacency matrix. The nested unordered maps also provide expected constant-time neighbour lookup by location key, although their iteration order is not deterministic.

## 8 Important Implementation Issues and Recommended Fixes

The following points distinguish what the supplied program currently does from what a real cab routing application would normally require.

| Issue | Why it matters | Recommended correction |
|---|---|---|
| Global spare-seat pool | Spare seats from an unrelated route can satisfy a request shortage. | Store and allocate spare capacity only for route segments on the selected path, or model spare vehicles at the network level with explicit repositioning. |
| Static active capacity | The same active seats remain available after every request. | Track reserved/occupied seats and release them when a trip completes or is cancelled. |
| Capacity checked after routing | Dijkstra may choose a cheap path whose capacity is inadequate even when another feasible path exists. | Incorporate capacity feasibility into path selection, or enumerate/compare feasible paths using a constrained shortest-path approach. |
| No handling of unmet passengers | When spare capacity is insufficient, the code still returns a fare and does not report unserved passengers. | Return a structured booking result containing assigned seats, unserved passengers, and acceptance status. |
| Unordered allocation order | The order in which spare entries are consumed is not a stable allocation policy. | Use a deterministic priority rule or a data structure representing capacity by route and operational priority. |

| Issue | Why it matters | Recommended correction |
|---|---|---|
| No input validation | Negative costs, negative passenger counts, malformed rows, and unknown locations are not handled robustly. | Validate each parsed field and reject invalid requests before running graph algorithms. |
| No file-open checks | A missing or unreadable input file can lead to an empty simulation without a clear error. | Check `infile.is_open()` and `outfile.is_open()` and report failures. |
| NODES section unused | The parser recognizes the header but does not register its listed nodes. | Parse node tokens explicitly, especially if isolated locations must be represented. |
| Missing explicit headers | `INT_MAX` and functional utilities should be included explicitly for portability. | Add `<climits>` and `<functional>`; compile with C++17 or later. |
| No request-level state | Requests are processed sequentially but there is no cab identity, trip duration, or vehicle position. | Introduce cab objects, trip lifecycle state, and a dispatch/availability layer. |
| No persistent state | Spare seats are only held in memory and disappear when the program exits. | Persist route capacity, reservations, and request records in a database or durable storage layer. |

### 8.1 A more reliable design direction

A stronger version could use the following sequence:

1. Validate the request and confirm that both locations exist.
2. Find feasible paths, considering both route cost and capacity.
3. Assign specific cabs or route-segment capacity to the request.
4. Update the available capacity immediately after a successful booking.
5. Calculate a transparent fare from route cost, passenger count, and any clearly defined demand/capacity rule.
6. Return an explicit accepted, partially fulfilled, or rejected status.
7. Store the booking and allocation so subsequent requests see the updated state.

## 9 Interview Questions and Suggested Answers

The answers below describe the implementation as it currently exists. They avoid claiming that the program performs live cab dispatch or maintains real-time vehicle occupancy.

### 9.1 Project overview and motivation

#### 9.1.1 Q1. Explain your project in one minute.

I developed a C++ cab-routing simulation that represents locations and routes as a weighted graph. Each route has a base cost, congestion cost, and active cab capacity. For every passenger request, I use Dijkstra's algorithm to find the minimum-cost path, check the bottleneck seat capacity along that path, and adjust the fare depending on whether the active capacity or configured spare seats can cover the request. The program reads requests from a text file and writes the selected paths and fares to an output file. The current version is a simulation, and route-specific spare-seat allocation and live cab tracking are important improvements.

#### 9.1.2 Q2. What problem are you trying to solve?

The project models how a routing system can choose a low-cost route while considering passenger capacity. It also demonstrates a simple fare adjustment when a request exceeds the active capacity available on the selected path.

#### 9.1.3 Q3. Why did you represent the city as a graph?

A city network naturally consists of locations connected by routes. Graphs let us represent those connections and attach a weight to each route. A shortest-path algorithm can then find the minimum-cost sequence of locations between a pickup and destination.

#### 9.1.4 Q4. Is your graph directed or undirected?

It is undirected in the current implementation. The `add_route()` function inserts both `graph[a][b]` and `graph[b][a]`, so a route can be traversed in either direction with the same cost and capacity.

### 9.2 Graph algorithms and data structures

#### 9.2.1 Q5. Which shortest-path algorithm did you use and why?

I used Dijkstra's algorithm because all route costs in the model are non-negative. It efficiently finds a minimum-cost path using a min-priority queue.

#### 9.2.2 Q6. How do you calculate the weight of an edge?

I add the base route cost and congestion cost. So each edge weight represents the combined cost of using that direct route.

#### 9.2.3 Q7. Why did you use a priority queue?

The priority queue helps select the unprocessed location with the smallest tentative distance. Using a min-priority queue avoids scanning every vertex to find that minimum on each iteration.

#### 9.2.4 Q8. What is the purpose of the parent map?

The distance map tells me the minimum cost to reach a location, while the parent map stores the previous location on the best-known path. I use the parent map to reconstruct the final path from the destination back to the source.

#### 9.2.5 Q9. What is the time complexity of Dijkstra in your implementation?

With an adjacency list and a binary-heap priority queue, the standard complexity is $O((V + E) \log V)$, where $V$ is the number of locations and $E$ is the number of routes.

#### 9.2.6 Q10. Why use an adjacency list instead of an adjacency matrix?

A road network is usually sparse, meaning each location connects to only a subset of all locations. An adjacency list stores only existing routes, so it uses less space than a full $V \times V$ matrix for sparse graphs.

## 9.3 C++ and object-oriented design

### 9.3.1 Q11. Why did you create an Edge structure?

The Edge structure groups all properties of a route, including base cost, congestion cost, active cab count, and seat capacity. This keeps route information organized and makes it easy to store each edge in the graph.

### 9.3.2 Q12. Why use unordered_map?

I used unordered maps for key-based lookup of locations and routes. They provide expected constant-time lookup and insertion, which is useful for building and querying the adjacency structure.

### 9.3.3 Q13. Why is a custom hash required?

The inactive-route map uses a pair of strings as its key. A standard hash for a pair is not provided as a general-purpose default in the C++ standard library, so I supplied a custom hash functor.

### 9.3.4 Q14. Why does make_key() sort the two endpoints?

The graph is undirected, so A–B and B–A represent the same route. Normalizing the pair ensures both orders map to the same inactive-route entry.

### 9.3.5 Q15. Which C++ standard does the code require?

The code uses structured bindings in range-based loops, so it requires C++17 or later.

## 9.4 Capacity and fare logic

### 9.4.1 Q16. How do you calculate active seat capacity?

The Edge constructor assumes four seats per cab. It multiplies the active cab count by four and stores the result in the edge's seat-capacity field.

### 9.4.2 Q17. Why do you take the minimum capacity along the path?

The minimum is the bottleneck capacity. In this simplified model, the passenger group must be supported on every route segment, so the segment with the fewest seats limits the total group size.

### 9.4.3 Q18. Explain your fare calculation.

If the passenger count is within the path's active capacity, I return the route cost. If there is a shortage but the spare-seat pool can cover it, I apply a 25% increase. If the pool is insufficient, I apply a 50% increase.

### 9.4.4 Q19. Are spare seats allocated to the selected route only?

Not in the current implementation. The code sums spare seats from all inactive routes and allocates from that global pool. A more realistic design should associate spare capacity with the route segments used by the selected path.

#### 9.4.5 Q20. Does the program update active seat capacity after a booking?

No. The current code consumes spare seats when they are needed, but it does not decrement active seats or track which cabs are assigned to a booking. Therefore, active capacity can be reused by later requests in the simulation.

#### 9.4.6 Q21. Is the returned fare multiplied by the number of passengers?

No. The current implementation calculates a route-level cost and applies a multiplier of 1.00, 1.25, or 1.50. It does not multiply the result by the passenger count.

### 9.5 Input/output and limitations

#### 9.5.1 Q22. How does the program read the input file?

It reads one line at a time using `getline()`, tracks which section is active, and uses `istringstream` to extract the fields for routes, inactive routes, and passenger requests.

#### 9.5.2 Q23. What happens when no path exists?

The shortest-path function returns an empty path when it cannot reconstruct a route to the destination. The fare function then returns -1, and the parser writes a failure message to the output file.

#### 9.5.3 Q24. What are the main limitations?

The main limitations are global rather than route-specific spare-seat allocation, no update of active capacity after a booking, shortest-path selection that ignores capacity, limited input validation, and no real-time tracking of cab locations or trip status.

#### 9.5.4 Q25. How would you improve the project?

I would integrate capacity constraints into route selection, allocate actual cabs to route segments, update availability after each booking, validate all input, and persist booking and vehicle states. I would also define a more transparent fare formula based on passenger count and operational demand.

#### 9.5.5 Q26. Can this be called a production cab-booking system?

Not yet. It is a C++ simulation of graph routing and capacity-based fare adjustment. A production system would also need live vehicle tracking, dispatch, booking state, concurrency handling, persistence, service APIs, and robust validation.

## 10 Interview Cheat Sheet

### 10.1 Key numbers to remember

| Item | Value in the supplied example |
|---|---|
| Locations listed | A, B, C, D |
| Direct routes | 4 undirected routes |
| Seats per active cab | 4 |
| Active capacity on A–B | 8 seats |
| Active capacity on A–C | 4 seats |
| Active capacity on B–C | 8 seats |
| Active capacity on C–D | 4 seats |
| Initial global spare-seat pool | 6 seats (4 on C–D and 2 on A–C) |
| Fare multipliers | 1.00, 1.25, and 1.50 |
| Shortest-path algorithm | Dijkstra |
| C++ standard | C++17 or later |

### 10.2 Quick vocabulary

- **Vertex / node:** A location in the network.
- **Edge:** A direct route between two locations.
- **Weight:** The combined base and congestion cost of a route.
- **Adjacency list:** A representation storing each vertex's neighbours.
- **Shortest path:** The path with the minimum sum of edge weights.
- **Priority queue:** A data structure that retrieves the smallest distance first in this implementation.
- **Bottleneck capacity:** The minimum active seat capacity across the selected path.
- **Spare capacity:** Additional seats configured for inactive routes.
- **Hash function:** A function that maps a key to a hash value for hash-table storage.

## 11 A Strong and Honest Final Project Pitch

I built a C++ cab-routing simulation using an undirected weighted graph. Each route stores a base cost, congestion cost, and active cab capacity. For every passenger request, the system uses Dijkstra's algorithm with a min-priority queue to find the minimum-cost route. It then checks the bottleneck seat capacity along that path and applies a fare multiplier depending on the availability of configured spare seats. I also implemented file-based input parsing and output reporting. The current version is a prototype: spare seats are pooled globally rather than allocated to matching route segments, and active cab occupancy is not tracked. My next step would be to make route selection capacity-aware and add actual cab assignment and booking-state management.

### 11.1 Final checklist before an interview

- Be able to explain the graph representation and why the graph is undirected.
- Explain why Dijkstra is suitable and state its non-negative-weight requirement.
- Trace one request through path selection, capacity checking, and fare calculation.
- Explain the purpose of `dist`, `parent`, the priority queue, and the custom hash.
- Clearly state that spare-seat allocation is global in the current code.

- Clearly state that active seat capacity is not decremented after a booking.
- Do not describe the simulation as a live dispatch or production booking platform.

# A Complete Supplied C++ Source Code

The following listing reproduces the supplied source file. It is included for reference; the technical review above discusses its current behavior and limitations.

```cpp
// Optimized cab_system.cpp with hash fix
#include <iostream>
#include <fstream>
#include <sstream>
#include <unordered_map>
#include <vector>
#include <string>
#include <queue>
#include <limits>
#include <algorithm>

using namespace std;

struct Edge {
    double base_cost;
    double congestion_cost;
    int cars;
    int seats;

    Edge(double base = 0, double congestion = 0, int cabs = 0)
        : base_cost(base), congestion_cost(congestion), cars(cabs), seats(cabs * 4) {}
};

// Custom hash for pair<string, string>
struct pair_hash {
    size_t operator()(const pair<string, string>& p) const {
        return hash<string>()(p.first) ^ (hash<string>()(p.second) << 1);
    }
};

pair<string, string> make_key(const string& a, const string& b) {
    return a < b ? make_pair(a, b) : make_pair(b, a);
}

class CabSystem {
private:
    unordered_map<string, unordered_map<string, Edge>> graph;
    unordered_map<pair<string, string>, int, pair_hash> inactive_routes;

public:
    void add_route(const string& a, const string& b, double base, double congestion, int cars) {
        Edge e(base, congestion, cars);
        graph[a][b] = e;
        graph[b][a] = e;
    }

    void set_inactive_route(const string& a, const string& b, int spare_seats) {
        inactive_routes[make_key(a, b)] = spare_seats;
    }

    pair<double, vector<string>> find_shortest_path(const string& src, const string& dest) {
        unordered_map<string, double> dist;
        unordered_map<string, string> parent;
        for (const auto& [node, _] : graph) dist[node] = numeric_limits<double>::max();
        dist[src] = 0;

```

```
        using PQ = pair<double, string>;
        priority_queue<PQ, vector<PQ>, greater<PQ>> pq;
        pq.emplace(0, src);

        while (!pq.empty()) {
            auto [cur_cost, u] = pq.top(); pq.pop();
            if (u == dest) break;

            for (const auto& [v, e] : graph[u]) {
                double cost = e.base_cost + e.congestion_cost;
                if (dist[v] > dist[u] + cost) {
                    dist[v] = dist[u] + cost;
                    parent[v] = u;
                    pq.emplace(dist[v], v);
                }
            }
        }

        vector<string> path;
        for (string curr = dest; curr != src; curr = parent[curr]) {
            if (parent.find(curr) == parent.end()) return {numeric_limits<double>::max(), {}};
            path.push_back(curr);
        }
        path.push_back(src);
        reverse(path.begin(), path.end());
        return {dist[dest], path};
    }

    int assign_spare_seats(int required) {
        int assigned = 0;
        for (auto& [route, spare] : inactive_routes) {
            int now = min(required - assigned, spare);
            spare -= now;
            assigned += now;
            if (assigned >= required) break;
        }
        return assigned;
    }

    double calculate_fare(const string& src, const string& dest, int P, vector<string>& path_used) {
        auto [base_cost, path] = find_shortest_path(src, dest);
        path_used = path;
        if (path.empty()) return -1;

        int seat_cap = INT_MAX;
        for (size_t i = 0; i + 1 < path.size(); ++i) {
            seat_cap = min(seat_cap, graph[path[i]][path[i + 1]].seats);
        }

        if (P <= seat_cap) return base_cost;

        int unfilled = P - seat_cap;
        int spare_total = 0;
        for (const auto& [_, spare] : inactive_routes) spare_total += spare;

        if (spare_total >= unfilled) {
            assign_spare_seats(unfilled);
            return 1.25 * base_cost;
        } else {
            assign_spare_seats(spare_total);
            return 1.5 * base_cost;
        }
    }
};

void process_input(CabSystem& cs, const string& infile_path, const string& outfile_path) {
```

```
    ifstream infile(infile_path);
    ofstream outfile(outfile_path);

    string line, section;
    while (getline(infile, line)) {
        if (line.empty()) continue;
        if (line.find("NODES:") == 0) { section = "nodes"; continue; }
        if (line.find("ROUTES:") == 0) { section = "routes"; continue; }
        if (line.find("INACTIVE:") == 0) { section = "inactive"; continue; }
        if (line.find("REQUESTS:") == 0) { section = "requests"; continue; }

        istringstream iss(line);
        string a, b; double base, cong; int val;

        if (section == "routes") {
            if (iss >> a >> b >> base >> cong >> val)
                cs.add_route(a, b, base, cong, val);
        } else if (section == "inactive") {
            if (iss >> a >> b >> val)
                cs.set_inactive_route(a, b, val);
        } else if (section == "requests") {
            int passengers;
            if (iss >> a >> b >> passengers) {
                vector<string> path;
                double fare = cs.calculate_fare(a, b, passengers, path);
                if (fare != -1) {
                    outfile << "Request: " << a << " -> " << b << " ("
                            << passengers << " passengers)\n";
                    outfile << "Path used: ";
                    for (size_t i = 0; i < path.size(); ++i) {
                        outfile << path[i];
                        if (i + 1 < path.size()) outfile << " -> ";
                    }
                    outfile << "\nFare: " << fare << "\n\n";
                } else {
                    outfile << "Request from " << a << " to " << b
                            << " failed. No valid path.\n\n";
                }
            }
        }
    }
}

int main() {
    CabSystem system;
    process_input(system, "input.txt", "output.txt");
    cout << "Simulation complete. Check output.txt for results." << endl;
    return 0;
}
```